

Yee's Scheme Derivation and TM Wave Results Replication

Overview

Starting from the Maxwell Equations, Yee's Scheme is derived and then applied to the Transverse Magnetic wave using an incident wave boundary condition. The results of the scheme are compared to Yee's results from the original 1966 paper [1]

Maxwell Equation Derivation

Yee's Scheme is based on the differential form of Maxwell's Equations:

$$\frac{\partial B}{\partial t} + \nabla \times E = 0$$

$$\frac{\partial D}{\partial t} - \nabla \times H = J$$

$$B = \mu H$$

$$D = \epsilon E$$

Corresponding to equations 1a-1D in Yee's original paper [1]. We start by analyzing the following equation:

$$\frac{\partial B}{\partial t} + \nabla \times E = 0$$

By using the cross product $\nabla \times E$, we can write the following:

$$\nabla \times E = \left[\begin{array}{ccc} \hat{i} & \hat{j} & \hat{k} \\ \frac{\partial}{\partial x} & \frac{\partial}{\partial y} & \frac{\partial}{\partial z} \\ E_x & E_y & E_z \end{array} \right] = \hat{i} \left(\frac{\partial E_z}{\partial y} - \frac{\partial E_y}{\partial z} \right) - \hat{j} \left(\frac{\partial E_z}{\partial x} - \frac{\partial E_x}{\partial z} \right) + \hat{k} \left(\frac{\partial E_y}{\partial x} - \frac{\partial E_x}{\partial y} \right)$$

The tangential components, $\frac{\partial E_x}{\partial x}$, $\frac{\partial E_y}{\partial y}$, and $\frac{\partial E_z}{\partial z}$, are all equal to zero and the cross product becomes:

$$\nabla \times E = \hat{i} \left(\frac{\partial E_z}{\partial y} - \frac{\partial E_y}{\partial z} \right) - \hat{j} \left(\frac{\partial E_z}{\partial x} - \frac{\partial E_x}{\partial z} \right) + \hat{k} \left(\frac{\partial E_y}{\partial x} - \frac{\partial E_x}{\partial y} \right)$$

The term, $\frac{\partial B}{\partial t}$ is a vector with component $\hat{i}$, $\hat{j}$, and $\hat{k}$.

$$\frac{\partial \vec{B}}{\partial t} = \hat{i} \left(\frac{\partial B_x}{\partial t} \right) + \hat{j} \left(\frac{\partial B_y}{\partial t} \right) + \hat{k} \left(\frac{\partial B_z}{\partial t} \right)$$

The first Maxwell Equation becomes:

$$\frac{\partial \vec{B}}{\partial t} + \nabla \times E = 0$$

$$\hat{i} \left(\frac{\partial B_x}{\partial t} \right) + \hat{j} \left(\frac{\partial B_y}{\partial t} \right) + \hat{k} \left(\frac{\partial B_z}{\partial t} \right) + \nabla \times E = \hat{i} \left(\frac{\partial E_z}{\partial y} - \frac{\partial E_y}{\partial z} \right) - \hat{j} \left(\frac{\partial E_z}{\partial x} - \frac{\partial E_x}{\partial z} \right) + \hat{k} \left(\frac{\partial E_y}{\partial x} - \frac{\partial E_x}{\partial y} \right) = 0$$

Which leads to three equations:

$$-- -- --$$

$$\hat{i} \rightarrow -\frac{\partial B_x}{\partial t} = \frac{\partial E_z}{\partial y} - \frac{\partial E_y}{\partial z}$$

$$\hat{j} \rightarrow -\frac{\partial B_y}{\partial t} = \frac{\partial E_x}{\partial z} - \frac{\partial E_z}{\partial x}$$

$$\hat{k} \rightarrow \frac{\partial B_z}{\partial t} = \frac{\partial E_x}{\partial y} - \frac{\partial E_y}{\partial x}$$

For the second Maxwell Equation:

$$\frac{\partial D}{\partial t} - \nabla \times H = J \text{ or } \frac{\partial D}{\partial t} = \nabla \times H - J$$

The cross product can be simplified as follows:

$$\nabla \times H = \nabla \times E = \begin{vmatrix} \hat{i} & \hat{j} & \hat{k} \\ \frac{\partial}{\partial x} & \frac{\partial}{\partial y} & \frac{\partial}{\partial z} \\ E_x & E_y & E_z \end{vmatrix} = \hat{i} \left(\frac{\partial H_z}{\partial y} - \frac{\partial H_y}{\partial z} \right) + \hat{j} \left(\frac{\partial H_z}{\partial x} - \frac{\partial H_x}{\partial z} \right) + \hat{k} \left(\frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial y} \right)$$

The J term is also a vector with $\hat{i}$, $\hat{j}$, and $\hat{k}$.

$$\hat{J} = \hat{i}(J_x) + \hat{j}(J_y) + \hat{k}(J_z)$$

Combining the terms:

$$\frac{\partial D}{\partial t} = \nabla \times H - J$$

$$\hat{i} \left(\frac{\partial D_x}{\partial t} \right) + \hat{j} \left(\frac{\partial D_y}{\partial t} \right) + \hat{k} \left(\frac{\partial D_z}{\partial t} \right) - \hat{i} \left(\frac{\partial H_z}{\partial y} - \frac{\partial H_y}{\partial z} \right) + \hat{j} \left(\frac{\partial H_z}{\partial x} - \frac{\partial H_x}{\partial z} \right) - \hat{k} \left(\frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial y} \right) = \hat{i}(J_x) + \hat{j}(J_y) + \hat{k}(J_z)$$

Simplifying, we now have 3 equations,

$$\hat{i} \rightarrow \frac{\partial D_x}{\partial t} = \frac{\partial H_z}{\partial y} - \frac{\partial H_y}{\partial z} - J_x$$

$$\hat{j} \rightarrow \frac{\partial D_y}{\partial t} = \frac{\partial H_x}{\partial z} - \frac{\partial H_z}{\partial x} - J_y$$

$$\hat{k} \rightarrow \frac{\partial D_z}{\partial t} = \frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial y} - J_z$$

And have a total of six equations that correspond with equations 2a through 2f of Yee's original paper [1].

$$-\frac{\partial B_x}{\partial t} = \frac{\partial E_z}{\partial y} - \frac{\partial E_y}{\partial z}$$

$$-\frac{\partial B_y}{\partial t} = \frac{\partial E_x}{\partial z} - \frac{\partial E_z}{\partial x}$$

$$\frac{\partial B_z}{\partial t} = \frac{\partial E_x}{\partial y} - \frac{\partial E_y}{\partial x}$$

$$\frac{\partial D_x}{\partial t} = \frac{\partial H_z}{\partial y} - \frac{\partial H_y}{\partial z} - J_x$$

$$\frac{\partial D_y}{\partial t} = \frac{\partial H_x}{\partial z} - \frac{\partial H_z}{\partial x} - J_y$$

$$\frac{\partial D_z}{\partial t} = \frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial y} - J_z$$

$$\frac{\partial D_z}{\partial t} = \frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial y} - J - z$$

For the Transverse Magnetic Wave (TM), we say the following are equivalent to zero since their electric fields align solely in the z-direction when propagating in the x-y plane.

$$E_z = E_y = 0, H_z = 0$$

From Yee's paper, and equations 2a through 2f, we can simplify the Maxwell Equations for the TM wave as follows:

$$\epsilon \frac{\partial E_z}{\partial t} = \frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial y}$$

$$\mu \frac{\partial H_z}{\partial t} = -\frac{\partial E_z}{\partial y}$$

$$\mu \frac{\partial H_y}{\partial t} = \frac{\partial E_z}{\partial x}$$

Numerical Discretization of Yee's Scheme

Yee discretized the continuous form of the Maxwell Equations outlined in the previous section by doing the following:

$$E_Z^{n+1}(i, j) = E_Z^n(i, j) - Z \frac{\Delta \tau}{\Delta x} \left[H_y^{n+\frac{1}{2}}[i + \frac{1}{2}, j] - H_y^{n+\frac{1}{2}}[i - \frac{1}{2}, j] \right] - Z \frac{\Delta \tau}{\Delta y} \left[H_x^{n+\frac{1}{2}}(i, j + \frac{1}{2}) - H_x^{n+\frac{1}{2}}(i, j - \frac{1}{2}) \right]$$

$$H_x^{n+\frac{1}{2}}(i, j + \frac{1}{2}) = H_x^{n-\frac{1}{2}}(i, j + \frac{1}{2}) - \frac{1}{Z} \frac{\Delta \tau}{\Delta y} [E_Z^n(i, j + 1) - E_Z^n(i, j)]$$

$$H_y^{n+\frac{1}{2}}(i + \frac{1}{2}, j) = H_y^{n-\frac{1}{2}}(i + \frac{1}{2}, j) + \frac{1}{Z} \frac{\Delta \tau}{\Delta x} [E_Z^n(i + 1, j) - E_Z^n(i, j)]$$

Where τ and Z are defined as follows:

$$\tau = ct = \sqrt{\frac{1}{\mu\epsilon}} t \text{ and } Z = \sqrt{\frac{\mu}{\epsilon}} = 376.7$$

Yee defined the grid of the scheme as follows:

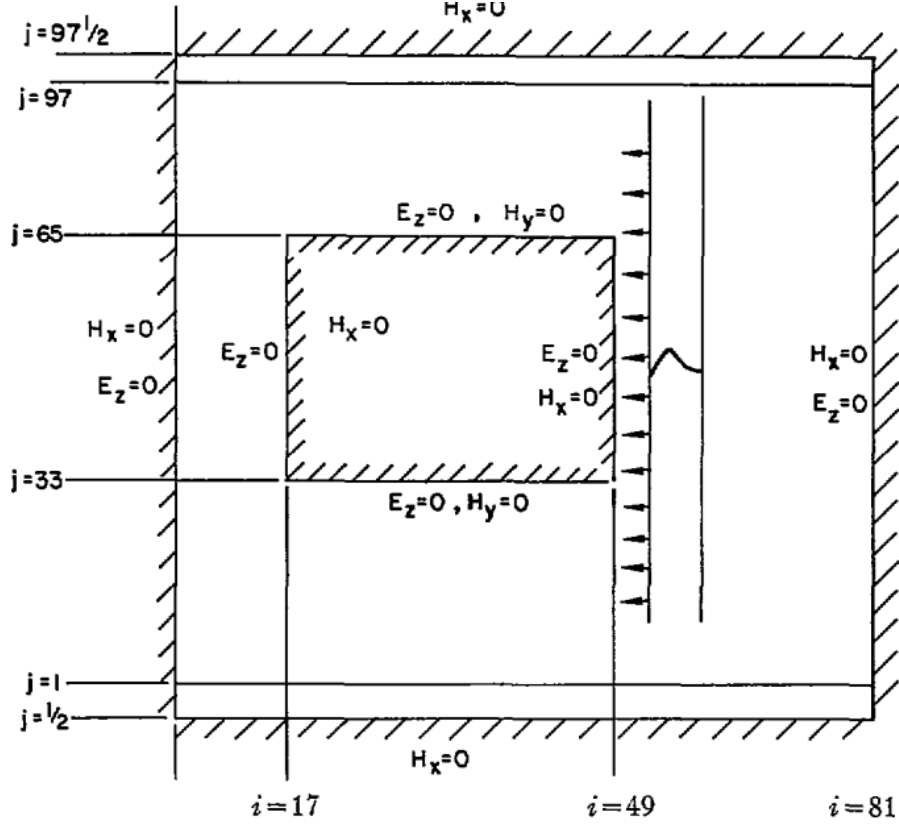


Fig. 2. Equivalent problem for scattering of a TM wave.

Where, the incident wave is applied to the right of the obstacle. The variables Δx and Δy were defined in relation to the width of the wave, α .

$$\Delta x = \Delta y = \frac{\alpha}{8}$$

Yee also relates $\Delta \tau$ and $c\Delta t$ to the width of the wave:

$$\Delta \tau = c\Delta t = \frac{\alpha}{16} \rightarrow \Delta t = \frac{\alpha}{16c}$$

We introduce a factor, f , that is designed to allow for refinement of the grid with minimal change to the algorithm.

$$\Delta x = \Delta y = \frac{\alpha}{8f} \quad \Delta \tau = \frac{\alpha}{16f} \quad \Delta t = \frac{\alpha}{16cf}$$

For the incident wave, Yee introduce the following equation and corresponding criteria:

$$E_z(x, y, n) = \sin \left[\frac{(x - 50\alpha + ct)\pi}{8\alpha} \right] \quad \text{for } 0 \leq x - 50\alpha + ct \leq 8\alpha$$

Which implies that the incident wave will occur at $i = 440$. To modify the wave such that it is applied within Yee's grid in figure 2, we introduce a variable to control the position, g , and width, f .

$$E_z(x, y, n) = \sin \left[\frac{(x - g\alpha + ct)\pi}{h\alpha} \right] \quad \text{for } 0 \leq x - g\alpha + ct \leq h\alpha$$

To ensure a stable solution, we refer to the following equation from Yee's paper:

$$\sqrt{(dx)^2 + (dy)^2} > c\Delta t$$

Which leads to the following inequality:

$$\frac{1}{4\sqrt{2}} > \frac{1}{16}$$

Which indicates stability for all values of α , f , g , and h . For the results, we choose values for the constants:

$$\alpha = 0.025$$

$$c = 3e8$$

$$f = 5$$

$$g = 1.5$$

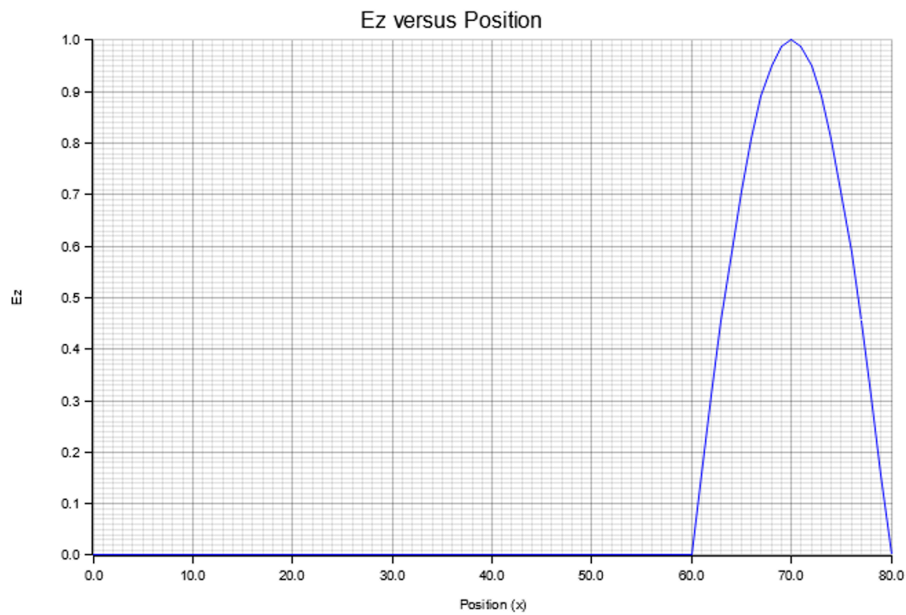
$$h = 0.5$$

Results

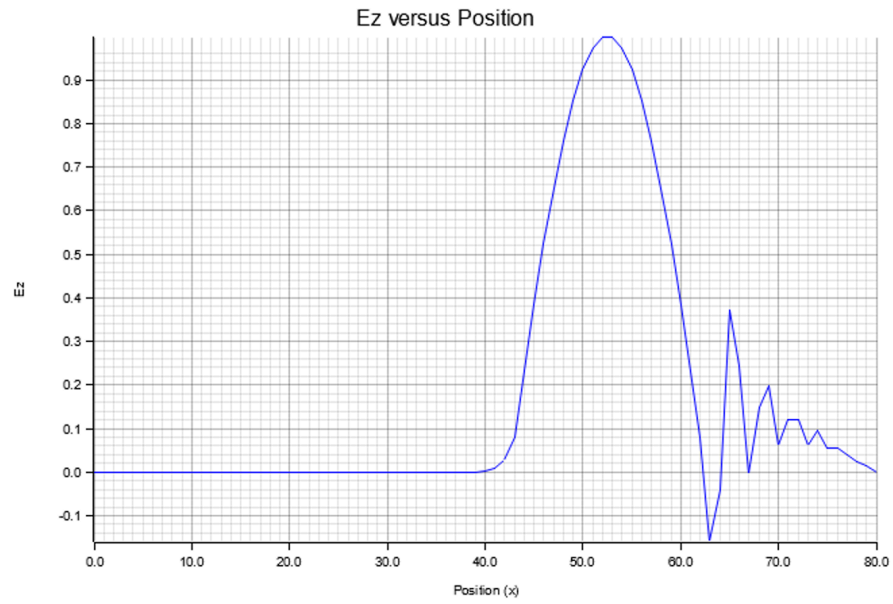
The numerical equations were implemented in an algorithm using Rust. The results are plotted for the no-obstacle and obstacle conditions at $j = 50$ and varying times, then compared with Yee's original results.

No Obstacle Conditions

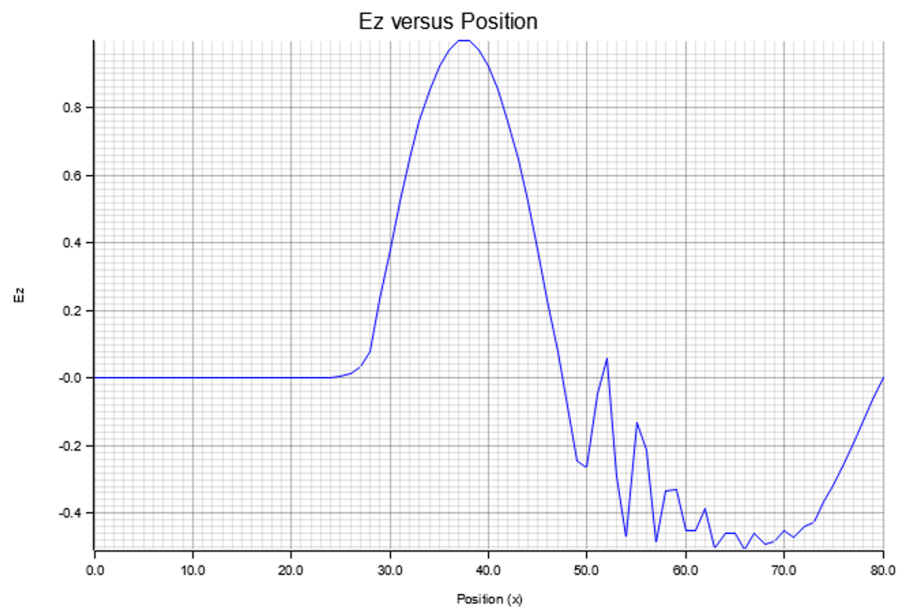
n=0



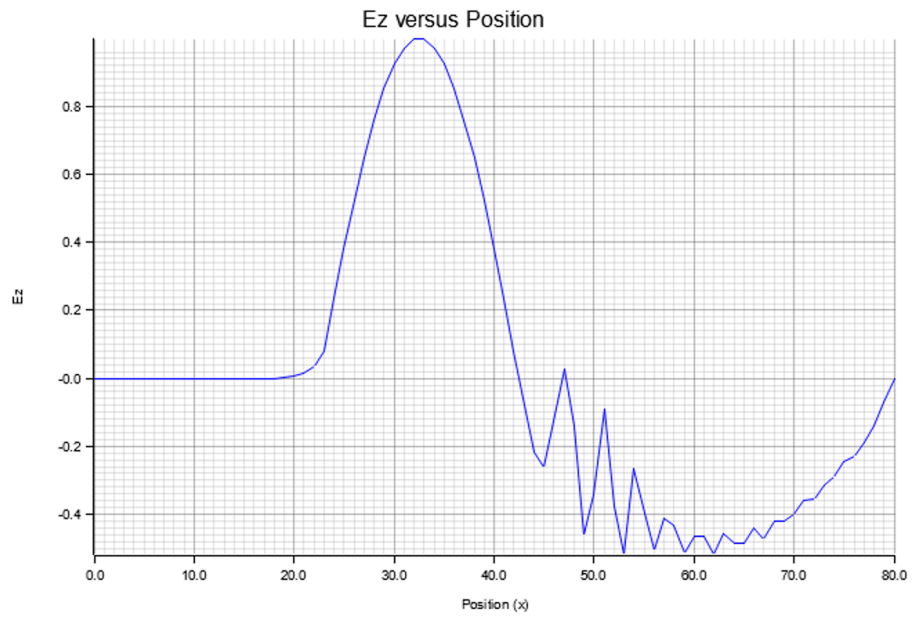
n=35



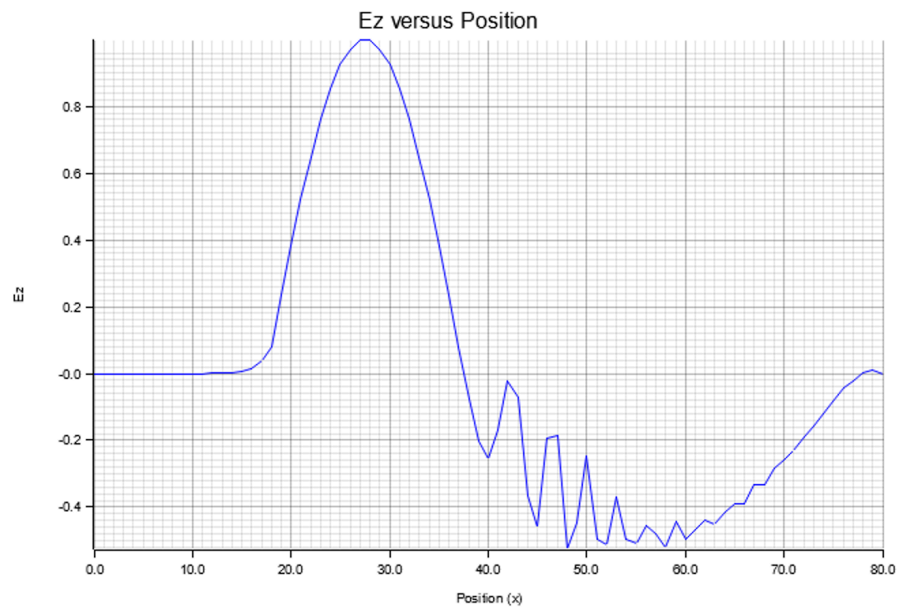
n=65



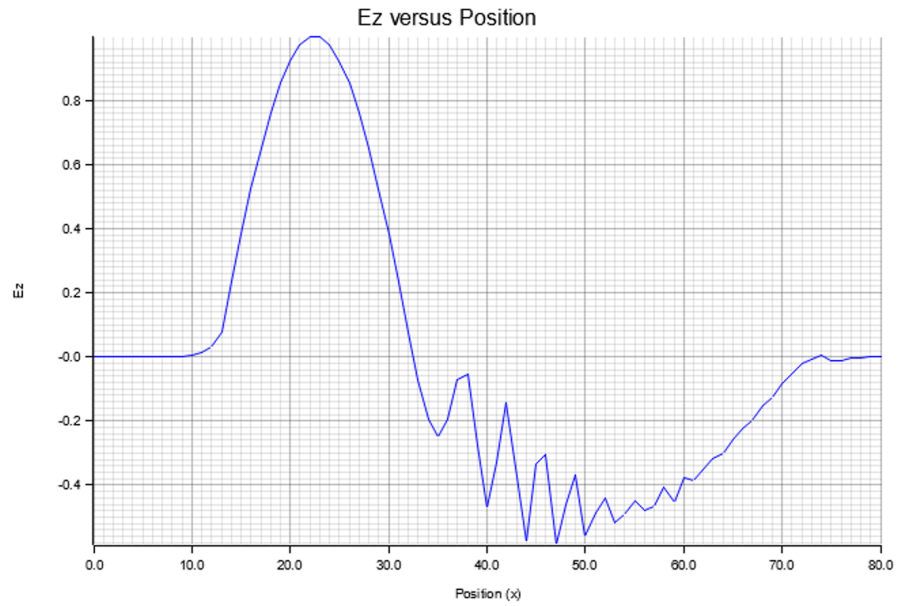
n=75



n=85

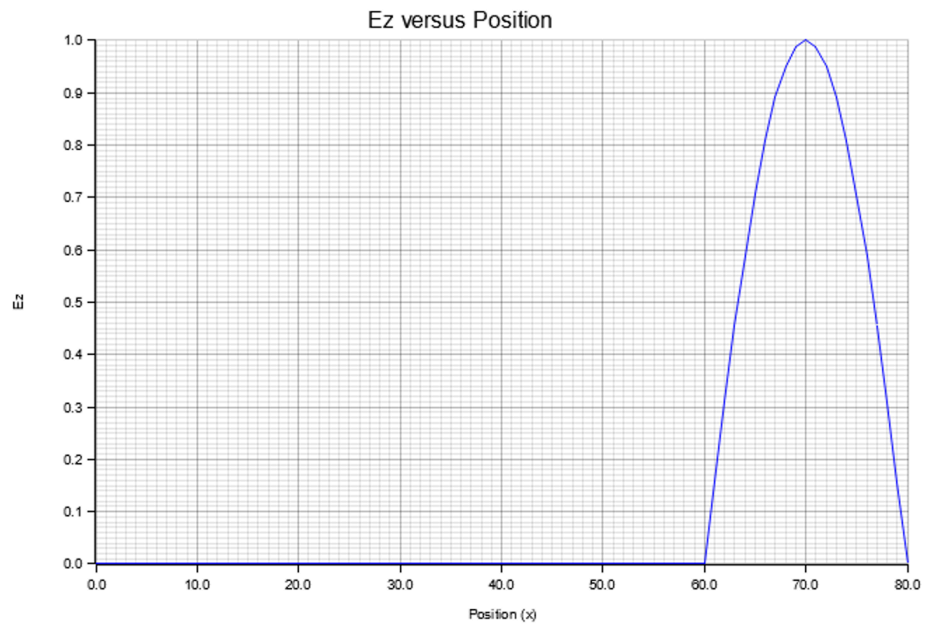


n=95

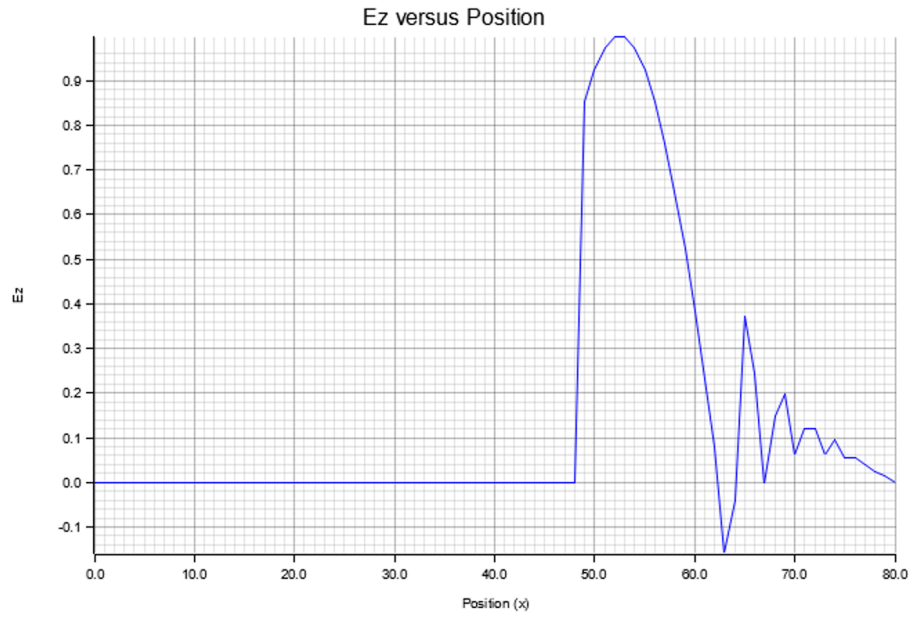


Obstacle Conditions

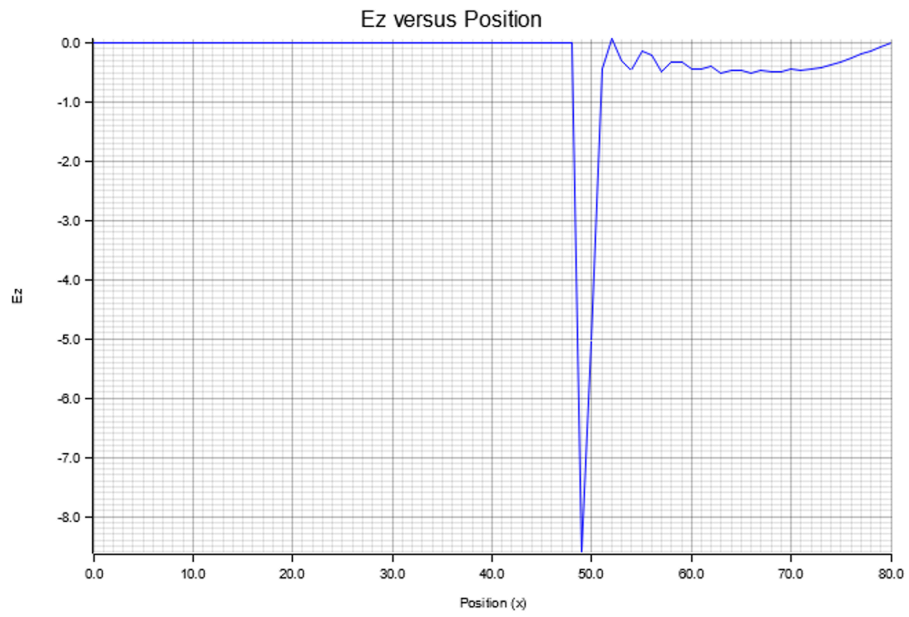
n=0



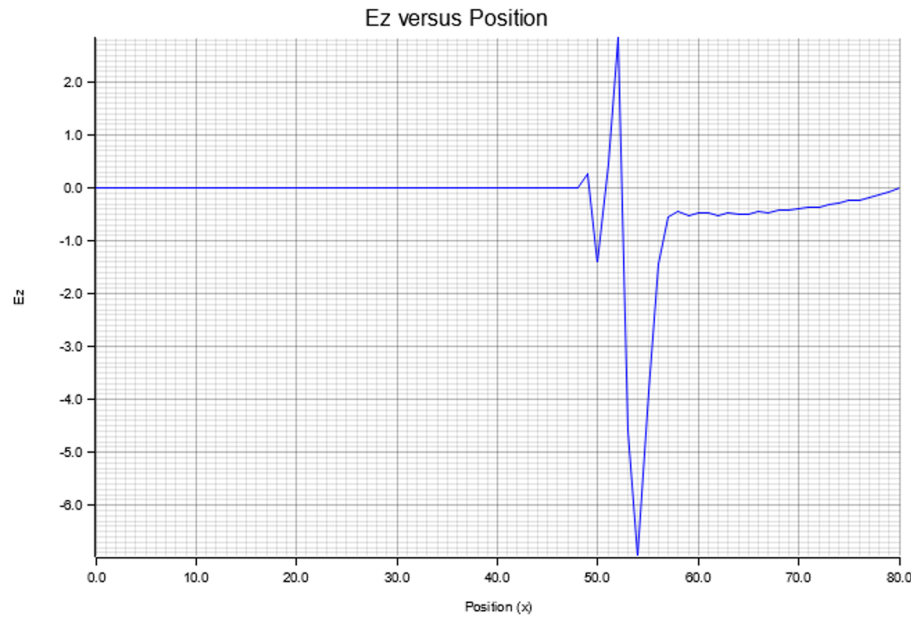
n=35



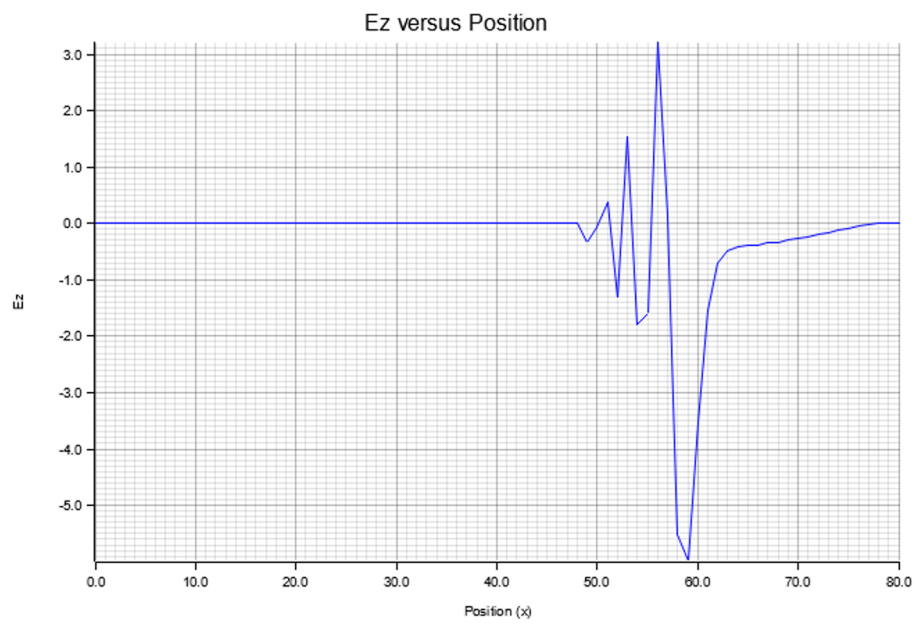
n=65



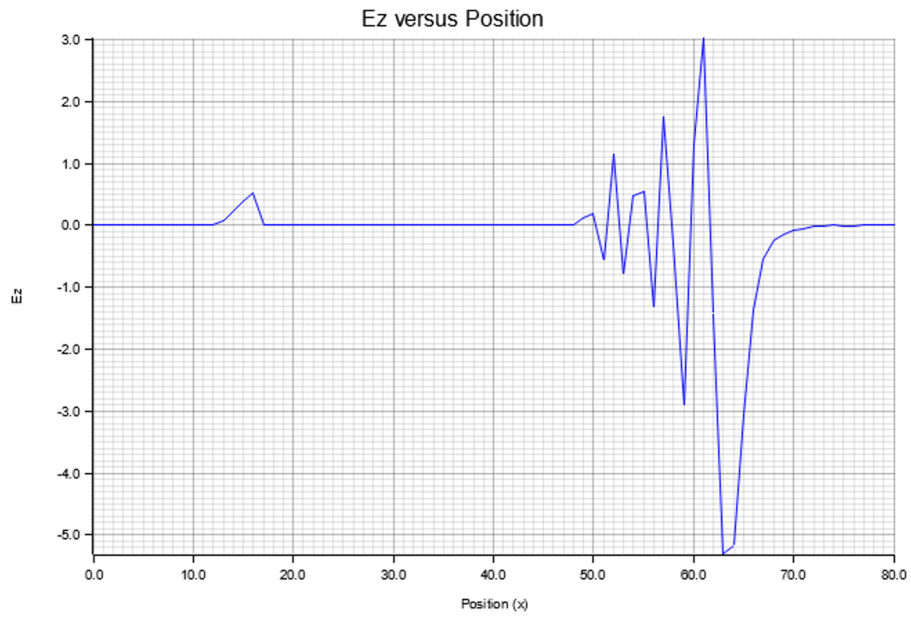
n=75



n=85

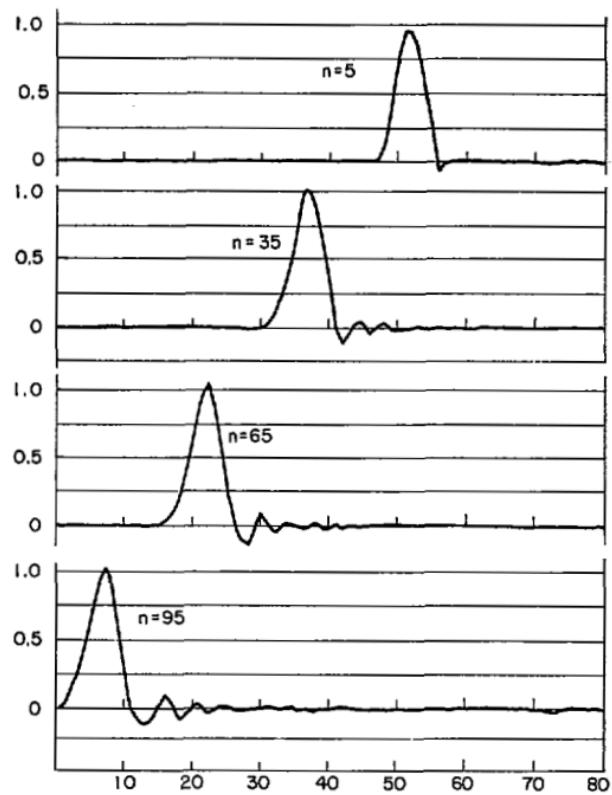


n=95

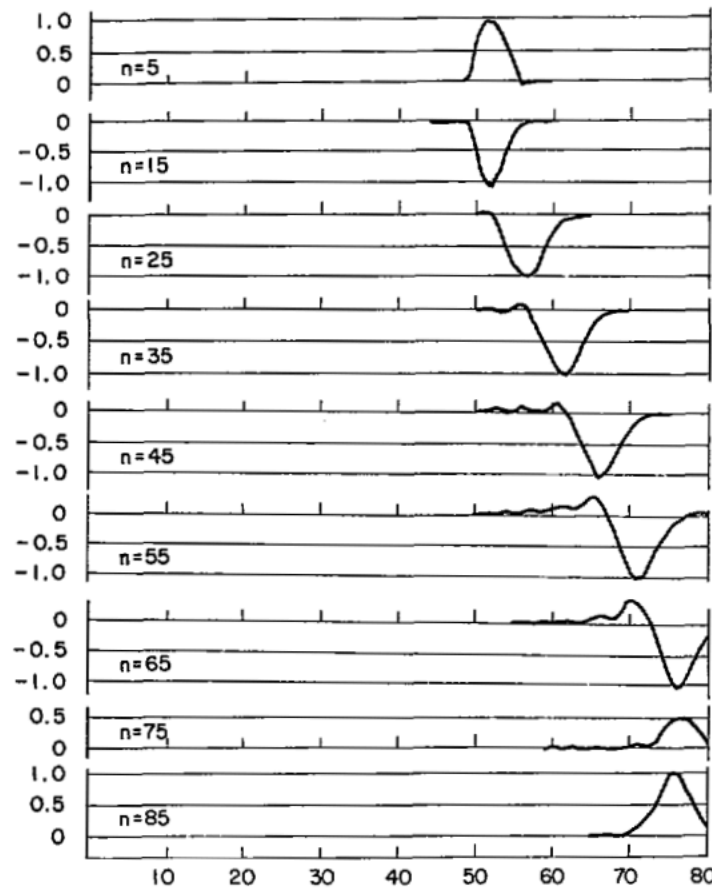


Yee's Original Results

No Obstacle



Obstacle



Conclusion

Overall, we derived equations Yee used for estimating the electric field for Transverse Magnetic wave with and without an obstacle. Since the exact results weren't published, we qualitatively conclude that results from the Rust algorithm correspond to the visual results from Yee's paper. In the no-obstacle conditions, the incident wave moves from left to right with respect to time. Then, when the obstacle is applied, scattering occurs from the reflecting.

Works Cited

[1] Kane Yee, "Numerical solution of initial boundary value problems involving maxwell's equations in isotropic media," in *IEEE Transactions on Antennas and Propagation*, vol. 14, no. 3, pp. 302-307, May 1966, doi: 10.1109/TAP.1966.1138693.

Appendix

No-Obstacle Main Code:

```
use nalgebra::{DMatrix};
use ndarray::Array3;
use std::f64::consts::PI;
use plotters::prelude::*;
```

```

use ndarray::s;

fn main() → Result<(), Box<dyn std::error::Error>> {
    // From yees paper
    let alpha: f64 = 0.025;

    // Resolution Factor
    let factor: f64=5.0;

    // Grid Partitions
    let dx: f64 = alpha / ((factor as f64)*8.0);
    let dy: f64 = alpha / ((factor as f64)*8.0);
    let dtau: f64 = alpha / ((factor as f64) *16.0);

    // Speed of light
    let cee=3e8;

    // E&M Constants
    let Zee: f64=376.7;
    //Time Partition
    let dt = (alpha)/(factor*16.0*cee);

    // Grid Dimensions (same as Yees)
    let nx: usize = 81;
    let ny: usize = 98;
    let nt: usize = 200;
    let xaxis: Vec<f64> = (0..nx).map(|i| i as f64).collect();

    // Hx and Hy are not plotted
    // They will be 2-dimension and not stored
    let Hxdata: Vec<f64> = vec![0.0; ny * nx];
    let mut Hx: DMatrix<f64> = DMatrix::from_row_slice(ny,nx, &Hxdata);

    let Hydata: Vec<f64> = vec![0.0; ny*nx];
    let mut Hy : DMatrix<f64>= DMatrix::from_row_slice(ny,nx, &Hydata);

    // Create 3-dimensional array for Ez
    // Will plot at varying locations on x and at different time, t
    let mut Ez: Array3<f64> = Array3::<f64>::zeros((nt,ny,nx));

    // Tuning parameters for incidental wave
    // Location
    let gee: f64=1.5;

    // Width
    let hhh: f64 =0.5;

    //Creating the initial conditions at t=0
    for i in 0..nx{
        let x: f64 = (i as f64) * dx;
        let aux: f64 = x - gee * alpha;
        for j in 0..ny{
            if (0.0 <= aux) && (aux <= (hhh * alpha)){
                Ez[[0,j,i]] = ((aux * PI)/(hhh*alpha)).sin();
            }
        }
    }
}

```

```

    }
  }
}

for n in 0..nt{
  let time =(n as f64)*dt;
  //Applying the incident wave first
  for i in 0..nx{
    let x = (i as f64) * dx;
    let aux = x- gee * alpha + cee * time;
    for j in 0..ny {
      if 0.0 <= aux && aux <= (hhh * alpha) {
        Ez[[n, j, i]] = ((aux * Pi) / (hhh * alpha)).sin();
      }
    }
  }
}

//Applying HX individually due to grid indexing
for i in 0..nx{
  for j in 0..ny-1{
    Hx[(j,i)] -= (1.0/Zee)*(dtau/dy)*(Ez[[n,(j+1),i]]-Ez[[n,j,i]]);
  }
}

//Applying HY individually due to grid indexing
for i in 0..(nx-1){
  for j in 0..ny{
    Hy[(j,i)] +=(1.0/Zee)*(dtau/dx)*(Ez[[n,j,(i+1)]]-Ez[[n,j,i]])
  }
}

//Calculating and storing the value for Ez to be plotted
for i in 1..(nx-1){
  for j in 1..(ny-1){
    if j+1<ny{
      Ez[[n+1,j,i]] = Ez[[n,j,i]]+Zee*(dtau/dx)*(Hy[(j,i)]-Hy[(j,(i-1))])-Zee*(dtau/dy)*(Hx[(j,i)]-Hx[(j-1,i)]);
    }
  }
}

// Edge Boundary Conditions
for j in 0..ny {
  Ez[[n+1,j, 0]] = 0.0;
  Ez[[n, j, (nx-1)]] = 0.0;
}

for i in 0..nx{
  Ez[[n+1,0, i]] = 0.0;
  Ez[[n, (ny-1), i]] = 0.0;
}

}
}

```

```

// Plotting
let n: i32 = 35;
let j: i32 = 50; // fixed j value
let row = Ez.slice(s![n, j, ..]).to_owned(); // shape: (nx,)

let (min_date, max_date) = xaxis
  .iter()
  .fold((f64::INFINITY, f64::NEG_INFINITY), |(min, max), d| {
    (min.min(*d), max.max(*d))
  });

let (min_alt, max_alt) = row
  .iter()
  .fold((f64::INFINITY, f64::NEG_INFINITY), |(min, max), a| {
    (min.min(*a), max.max(*a))
  });

// Set up drawing backend
let root = BitMapBackend::new("No-Obstacle.png", (700,500)).into_drawing_area();
root.fill(&WHITE)?;

let mut chart = ChartBuilder::on(&root)
  .caption("Ez versus Position", ("sans-serif", 20))
  .margin(30)
  .x_label_area_size(40)
  .y_label_area_size(60)
  .build_cartesian_2d(min_date..max_date, min_alt..max_alt)?;

chart.configure_mesh()
  .x_desc("Position (x)")
  .y_desc("Ez")
  .draw()?;

let cleaned_data: Vec<(f64, f64)> = xaxis
  .iter()
  .zip(row.iter())
  .map(|(&x, &y)| (x, y))
  .collect();

chart.draw_series(LineSeries::new(cleaned_data, &BLUE))?;

Ok(())
}

```

No-Obstacle Cargo.toml:

```

[package]
name = "my_project"
version = "0.1.0"
edition = "2024"

```

```
[dependencies]
nalgebra = "0.32" # or the latest version
rusqlite = { version = "0.31.0", features = ["bundled"] }
plotters = "0.3"
plotters-svg = "0.3"
ndarray = "0.16.1"
```

Obstacle Main Code:

```
use nalgebra::DMatrix;
use ndarray::Array3;
use std::f64::consts::PI;
use plotters::prelude::*;
use ndarray::s;

fn main() → Result<(), Box<dyn std::error::Error>> {
    // From yees paper
    let alpha: f64 = 0.025;

    // Resolution Factor
    let factor: f64=5.0;

    // Grid Partitions
    let dx: f64 = alpha / ((factor as f64)*8.0);
    let dy: f64 = alpha / ((factor as f64)*8.0);
    let dtau: f64 = alpha / ((factor as f64) *16.0);

    // Speed of light
    let cee=3e8;

    // E&M Constants
    let Zee: f64=376.7;
    //Time Partition
    let dt = (alpha)/(factor*16.0*cee);

    // Grid Dimensions (same as Yees)
    let nx: usize = 81;
    let ny: usize = 98;
    let nt: usize = 200;
    let xaxis: Vec<f64> = (0..nx).map(|i| i as f64).collect();

    // Hx and Hy are not plotted
    // They will be 2-dimension and not stored
    let Hxdata: Vec<f64> = vec![0.0; ny * nx];
    let mut Hx: DMatrix<f64> = DMatrix::from_row_slice(ny,nx, &Hxdata);

    let Hydata: Vec<f64> = vec![0.0; ny*nx];
    let mut Hy : DMatrix<f64>= DMatrix::from_row_slice(ny,nx, &Hydata);

    // Create 3-dimensional array for Ez
    // Will plot at varying locations on x and at different time, t
    let mut Ez: Array3<f64> = Array3::<f64>::zeros((nt,ny,nx));
```



```

// Tuning parameters for incidental wave
// Location
let gee: f64=1.5;

// Width
let hhh: f64 =0.5;

//Creating the initial conditions at t=0
for i in 0..nx{
  let x: f64 = (i as f64) * dx;
  let aux: f64 = x - gee * alpha;
  for j in 0..ny{
    if (0.0 <= aux) && (aux <= (hhh * alpha)){
      Ez[[0,j,i]] = ((aux * Pi)/(hhh*alpha)).sin();
    }
  }
}

// Obstacle boundary conditions
for i in 17..49{
  for j in 33..65{
    Ez[[0,j,i]] = 0.0;
  }
}

for n in 0..(nt-1){
  let time =(n as f64)*dt;
  //Applying the incident wave first
  for i in 0..nx{
    let x = (i as f64) * dx;
    let aux = x- gee * alpha + cee * time;
    for j in 0..ny {
      if 0.0 <= aux && aux <= (hhh * alpha) {
        Ez[[n, j, i]] = ((aux * Pi) / (hhh * alpha)).sin();
      }
    }
  }

  //Applying HX individually due to grid indexing
  for i in 0..nx{
    for j in 0..ny-1{
      Hx[[j,i]] -= (1.0/Zee)*(dtau/dy)*(Ez[[n,(j+1),i]]-Ez[[n,j,i]]);
    }
  }

  //Applying HY individually due to grid indexing
  for i in 0..(nx-1){
    for j in 0..ny{
      Hy[[j,i]] +=(1.0/Zee)*(dtau/dx)*(Ez[[n,j,(i+1)]]-Ez[[n,j,i]])
    }
  }

  //Calculating and storing the value for Ez to be plotted

```

```

for i in 1..(nx-1){
  for j in 1..(ny-1){
    if j+1<ny{
      Ez[[n+1,j,i]] = Ez[[n,j,i]]+Zee*(dtau/dx)*(Hy[(j,i)]-Hy[(j,i-1)])-Zee*(dtau/dy)*(Hx[(j,i)]-Hx[(j-1,i)]);
    }
  }
}

// Edge Boundary Conditions
for j in 0..ny {
  Ez[[n+1,j, 0]] = 0.0;
  Ez[[n, j, (nx-1)]] = 0.0;
}

for i in 0..nx{
  Ez[[n+1,0, i]] = 0.0;
  Ez[[n, (ny-1), i]] = 0.0;
}

// Obstacle Boundary conditions
for i in 17..49{
  for j in 33..65{
    Ez[[n+1,j,i]] = 0.0;

  }
}

}

// Plotting
let n: i32 = 0;
let j: i32 = 50; // fixed j value
let row = Ez.slice(s![n, j, ..]).to_owned(); // shape: (nx,)

let (min_date, max_date) = xaxis
  .iter()
  .fold((f64::INFINITY, f64::NEG_INFINITY), |(min, max), d| {
    (min.min(*d), max.max(*d))
  });

let (min_alt, max_alt) = row
  .iter()
  .fold((f64::INFINITY, f64::NEG_INFINITY), |(min, max), a| {
    (min.min(*a), max.max(*a))
  });

// Set up drawing backend
let root = BitMapBackend::new("No_Obstacle.png", (700,500)).into_drawing_area();
root.fill(&WHITE)?;

let mut chart = ChartBuilder::on(&root)
  .caption("Ez versus Position", ("sans-serif", 20))
  .margin(30)
  .x_label_area_size(40)
  .y_label_area_size(60)

```

```

        .build_cartesian_2d(min_date..max_date, min_alt..max_alt)?;

chart.configure_mesh()
  .x_desc("Position (x)")
  .y_desc("Ez")
  .draw()?;

let cleaned_data: Vec<(f64, f64)> = xaxis
  .iter()
  .zip(row.iter())
  .map(|(&x, &y)| (x, y))
  .collect();

chart.draw_series(LineSeries::new(cleaned_data, &BLUE))?;

Ok(())
}

```

Obstacle Cargo.toml:

```

[package]
name = "my_project"
version = "0.1.0"
edition = "2024"

[dependencies]
nalgebra = "0.32" # or the latest version
rusqlite = { version = "0.31.0", features = ["bundled"] }
plotters = "0.3"
plotters-svg = "0.3"
ndarray = "0.16.1"

```